

Formal Mathematical Formulation of Sudoku Pattern and Digit Optimization Problems

1 Basic Definitions

Let the 9×9 Sudoku grid be indexed by cells

$$i \in \{1, 2, \dots, 81\}, \quad (r, c), \quad r, c \in \{1, \dots, 9\}.$$

1.1 Full Solution Grids

Let $\mathcal{G} = \{G_1, \dots, G_n\}$ denote a set of fully solved Sudoku grids. Each G_k is a 9×9 array with entries in $\{1, \dots, 9\}$.

1.2 Pattern (Mask)

A Sudoku *pattern* is a binary vector:

$$m \in \{0, 1\}^{81}.$$

Here $m_i = 1$ means the value in cell i is a given clue; $m_i = 0$ means it is blank.

1.3 Induced Puzzle

Given m and a full solution G_k , the puzzle is:

$$P(m, G_k)[i] = \begin{cases} G_k[i] & \text{if } m_i = 1, \\ 0 & \text{if } m_i = 0. \end{cases}$$

1.4 Solution Set

Let $S(P)$ be the set of valid completions of puzzle P . A puzzle is uniquely solvable iff $|S(P)| = 1$.

1.5 Difficulty Score

We define a difficulty functional $\text{Diff}(P)$ as

$$\text{Diff}(P) = \alpha \cdot \text{TechDiff}(P) + \beta \cdot \text{BranchDifficulty}(P) + \gamma \cdot \sum_{i=1}^{81} \log(|C_i(P)|),$$

where $C_i(P)$ are candidate sets during logical solving.

2 Problem A: Pattern Optimization

Summary (Plain Language)

Problem A asks:

“Among all possible clue patterns (which cells are revealed), find a *single universal pattern* that produces uniquely solvable and consistently difficult puzzles for *every* solved grid in our sample set.

In other words, find a general puzzle *template* that stays hard no matter which digit arrangement is used.”

This problem discovers structural Sudoku patterns that are robust across many solutions. Such patterns produce stable puzzle difficulty, aesthetic symmetry, and form the backbone of a puzzle generator.

2.1 Constraints

For all $k = 1, \dots, n$:

$$|S(P(m, G_k))| = 1,$$

$$\text{Diff}(P(m, G_k)) \geq D_{\min}.$$

Optional clue-count:

$$C_{\min} \leq \sum m_i \leq C_{\max}.$$

2.2 Objective (Robust Difficulty)

Define

$$D_{\min}(m) = \min_{k=1, \dots, n} \text{Diff}(P(m, G_k)).$$

The optimization problem is:

$$\begin{array}{ll} \max_{m \in \{0,1\}^{81}} & D_{\min}(m) \\ \text{s.t.} & |S(P(m, G_k))| = 1 \quad \forall k, \\ & \text{Diff}(P(m, G_k)) \geq D_{\min} \quad \forall k, \\ & C_{\min} \leq \sum m_i \leq C_{\max}. \end{array}$$

3 Problem B: Digit Optimization Under a Fixed Pattern

Summary (Plain Language)

Problem B assumes the clue pattern m is already chosen (for style, symmetry, or theme). Then it asks:

“Given this fixed pattern, which full Sudoku solution grid G produces the *hardest* possible puzzle while still remaining uniquely solvable?”

This is the engine used by puzzle apps: one pattern can generate thousands of puzzles by searching for digit layouts that maximize difficulty.

3.1 Digit Optimization Problem

$$\begin{array}{ll} \max_{G \in \mathcal{G}} & \text{Diff}(P(m, G)) \\ \text{s.t.} & |S(P(m, G))| = 1. \end{array}$$

4 Conclusion

Problem A identifies **structurally strong Sudoku patterns** that remain robustly difficult across multiple solutions. Problem B **fills those patterns with the best digits** to generate many unique, high-quality puzzles. Together, they form a complete mathematical framework for professional Sudoku puzzle generation.